ECE 5159

DIGITAL SYSTEM DESIGN WITH VHDL

Engr. Professor Alice Oluwafunke Oke

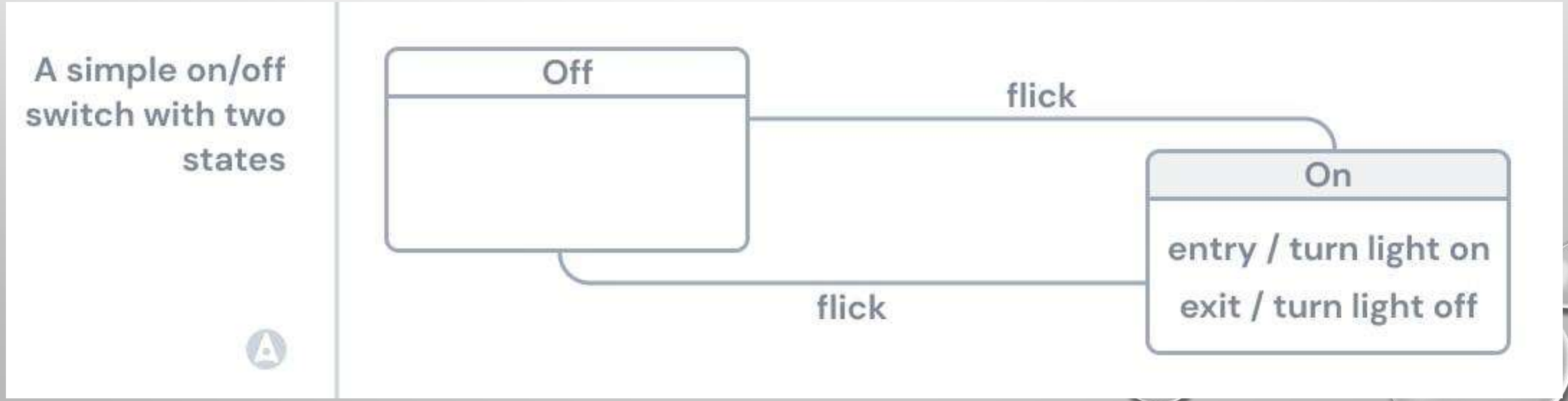
Department of Computer Engineering, ACU, Oyo

STATE MACHINE

- A state machine models system behaviour
- Specifically, systems with a finite number of states, hence, they are also called Finite State Machine (FSM).
- A state machine can be considered as a set of signifiers (active/inactive, open/closed, on/off, etc
- FSMs have a start state, end state, and any number of intermediate states.
- The system transitions from one state to another based on a condition or triggered event.

STATE MACHINE

- The term "state" is defined as the condition or status of a system that is ready to execute a transition.
- In response to some inputs, the FSM can transition from one state to another; this change is referred to as a transition.



STATE MACHINE

A state machine is a behavior model

It consists of

- a finite number of states and is therefore also called finite-state machine (FSM).
- Is based on the current state (initial state)
- a given input
- the machine performs state transitions and
- produces outputs.

TYPES OF STATE MACHINE

There are two types of basic state machines:

- Deterministic Finite State Machine
- Non-Deterministic State Machines

1. Deterministic Finite State Machine

They have a fixed order of events.

They are also called chronological state machines or order-dependent systems.

Consider a traffic light. The light has three stages: red, yellow, and green.

The light changes from green to yellow to red in a set order. The light is in only one state at any given time (the light can not be both green and red).

TYPES OF STATE MACHINE

2. Non-Deterministic State Machines

Non-deterministic state machines allow flexibility. The order of events changes each time the machine executes. Take a real-world example of an elevator.

The elevator has no fixed order to reach a final state. It determines where it goes each time a person presses the button for a specific floor. Non-deterministic state machines are useful to model concurrency, parallelism, or shared state.

STATE MACHINE USE CASES

State machines are helpful for a variety of purposes. They

- can be used to model the flow of logic within a program
- represent the states of a system

Modeling Business Workflow Processes

State machines are ideal for modeling business workflows. This includes:

- account setup flows,
- order completion, or
- a hiring process.

STATE MACHINE USE CASES Contd

Business Decision-Making

Companies pair FSMs with their data strategy to explore the cause and effect of business scenarios to make informed business decisions.

Software Design

Developers use state machines to model how a user interacts with the application. These models are also helpful in modeling business logic flow in the system. State machines help developers ensure their software is efficient, scalable, and maintainable

FINITE STATE MACHINE MODELS

The basic types are

- Moore machines
- Mealy machines

Complex types

- Harel and
- UML statecharts.

STATE MACHINE Contd

The basic building blocks are

- states and
- transitions.

A state is a situation of a system depending on previous inputs and causes a reaction on following inputs. One state is marked as the initial state; this is where the execution of the machine starts.

A state transition defines for which input a state is changed from one to another. Depending on the state machine type, states and/or transitions produce outputs.

STATE MACHINE Contd

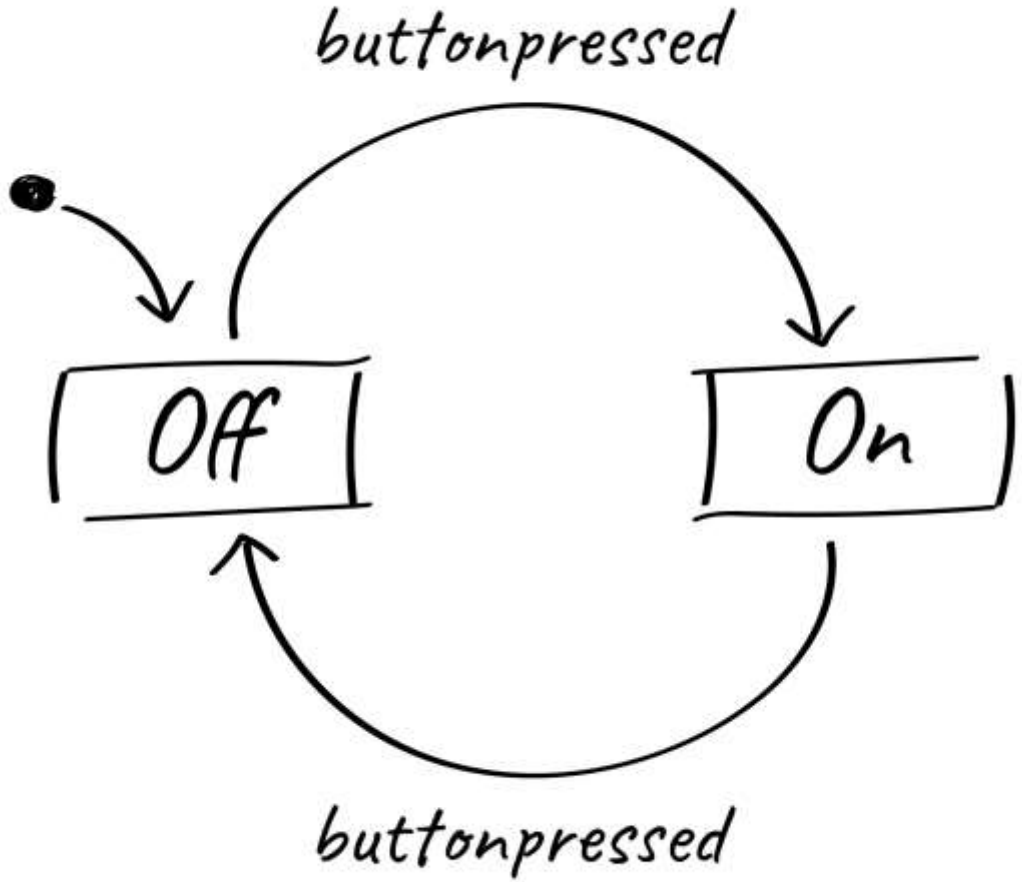


Figure 1

Consider Figure 1

It consists of two states: *Off* and *On*.

***On* is the initial state here and activated when the state machine is executed.**

The arrows between the states are state transitions. They define for which input a state change occurs.

Here, the active state is changed from *On* to *Off* for the input *buttonpressed*, and back again to *On* for the same input.

STATE MACHINE Contd

In automata theory an automaton reacts on inputs and produces outputs.

where, the terms input and output are usually used for symbols which belong to an alphabet.

Modern state machines use an extended definition of inputs and outputs.

STATE MACHINE Contd

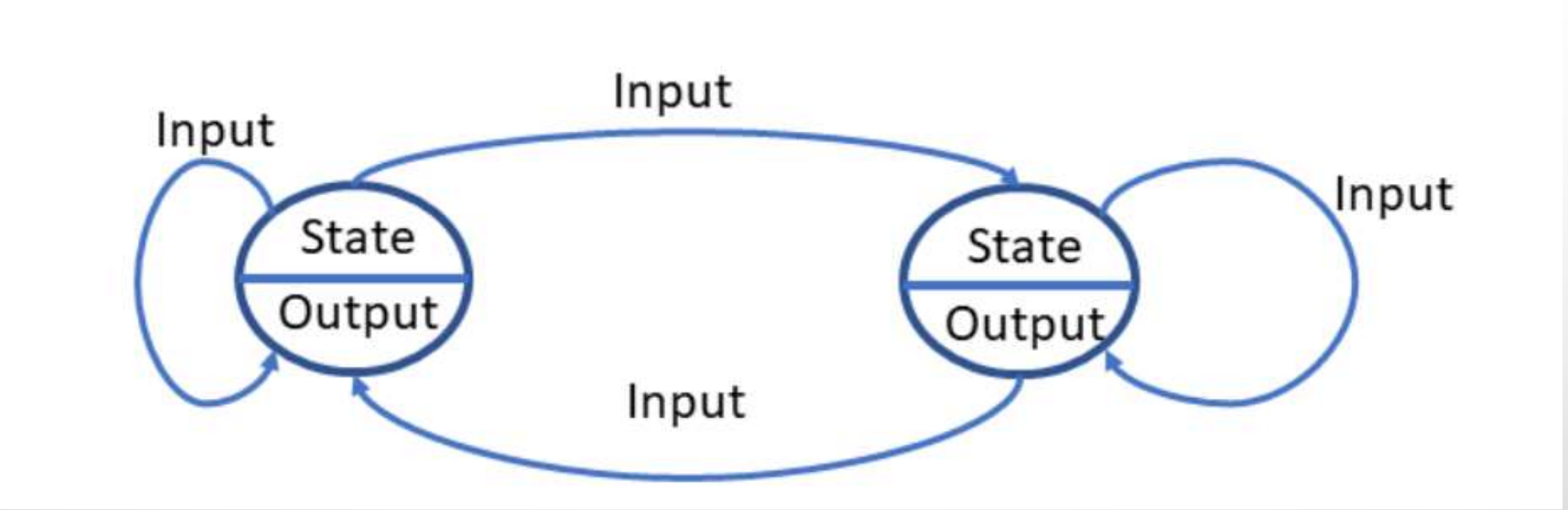
Inputs can be events like

- a button click or
- a time trigger

Outputs are actions like

- an operation call or
- a variable assignment.

Moore Machine Model



Generic Moore Model

Moore Machine

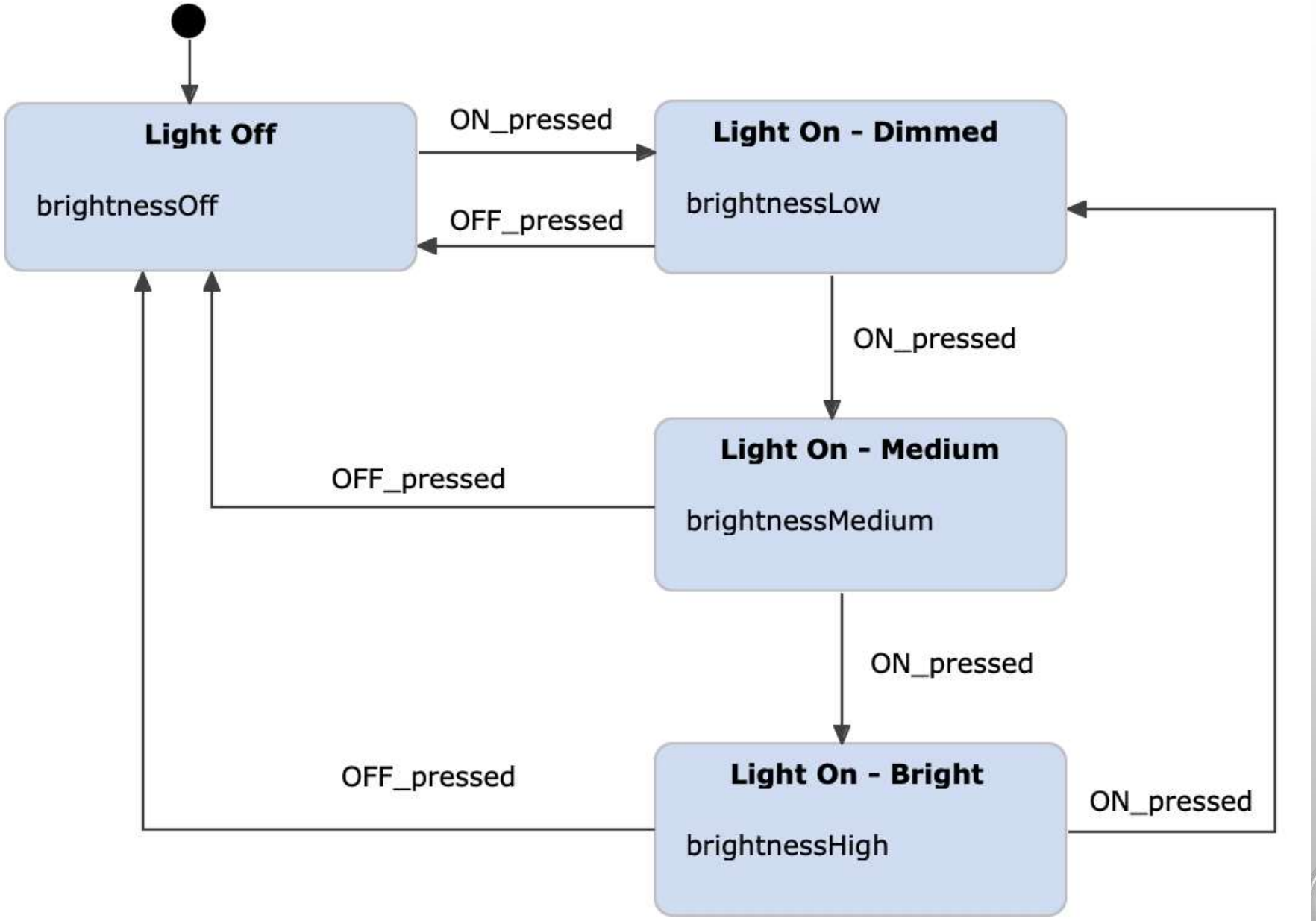
Moore Machine Concept:

- In this state machine, the output is determined only by the present active state of the state machine and not by any input events.
- No output during state transition.
- Output is represented inside the state.
- ‘Output’ is also called ‘Action.’
- In the Moore model, the ‘Output’ is also called ‘Entry action.’

Moore Machine Characteristics:

- Outputs depend only on the current state.
- Outputs change synchronously at the beginning of each state.
- The output is associated with the states.

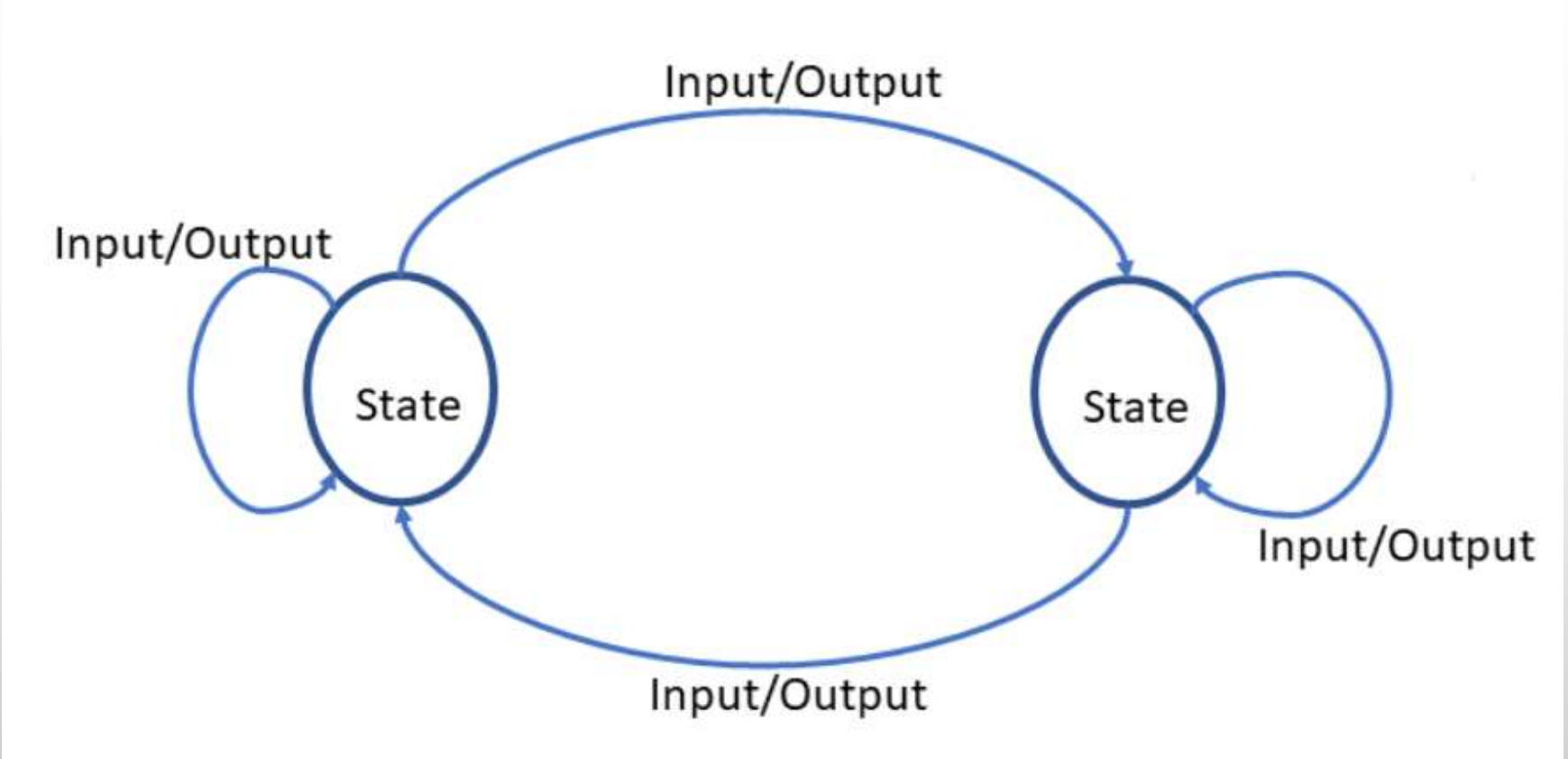
Moore Machine Example



Moore State Transition Table

State	Entry actions (output)	Next state	
		Input events	
		OFF	ON
Off	Light off	---ignored--	Dim
Dim	Make light dim	Off	Medium
Medium	Make light medium	Off	Bright
Bright	Make light bright	Off	Dim

Mealy Machine



Generic Mealy Model

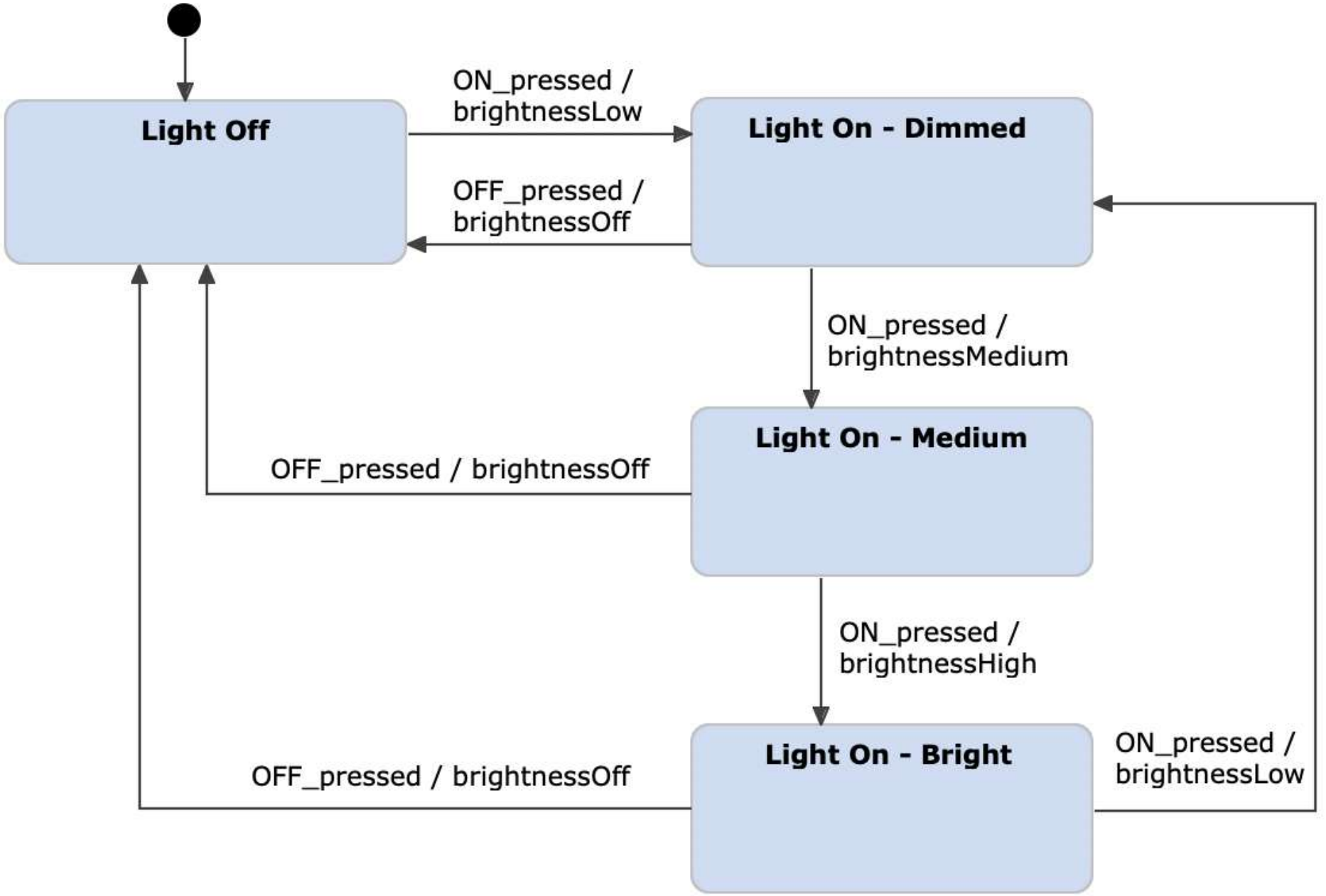
Mealy Machine Contd

Mealy Machine Concept:

- In this State machine, the output is represented along with each input separated by a ‘/’ character.
- An ‘Output’ is also called ‘Actions.’
- In the Mealy model, the ‘Output’ is also called ‘Input action.’ Because when the input is received, the action may happen or output happens, and there will be a state transition.

In the mealy machine, the output is not produced inside the states but along the transition.

Mealy Machine Example



Mealy State Transition Table

Present state	Next state			
	Input events			
	OFF		ON	
	Next state	input action (output)	Next state	input action (output)
Off	off	--ignored--	Dim	Make light dim
Dim	Off	Light off	Medium	Make light medium
Medium	Off	Light off	Bright	Make light bright
Bright	Off	Light off	Dim	Make light dim

Mealy State Transition Table Contd

State transition table
Light control Mealy machine

Present state	Next state			
	Input events			
	OFF		ON	
	Next state	input action (output)	Next state	input action (output)
Off	off	--ignored--	Dim	Make light dim
Dim	Off	Light off	Medium	Make light medium
Medium	Off	Light off	Bright	Make light bright
Bright	Off	Light off	Dim	Make light dim

State transition table
Light control Mealy machine

Present state	Next state			
	Input events			
	OFF		ON	
	Next state	input action (output)	Next state	input action (output)
Off	off	--ignored--	Dim	Make light dim
Dim	Off	Light off	Medium	Make light medium
Medium	Off	Light off	Bright	Make light bright
Bright	Off	Light off	Dim	Make light dim

Harel Machine

Mealy machines can reduce the number of required states, but for complex systems such automatons get easily unmanageable.

Harel concluded that " *a state approach must be*

- *modular,*
- *hierarchical and*
- *well-structured*

So, he introduced additional concepts like state composition and orthogonality.

Harel Machine Contd

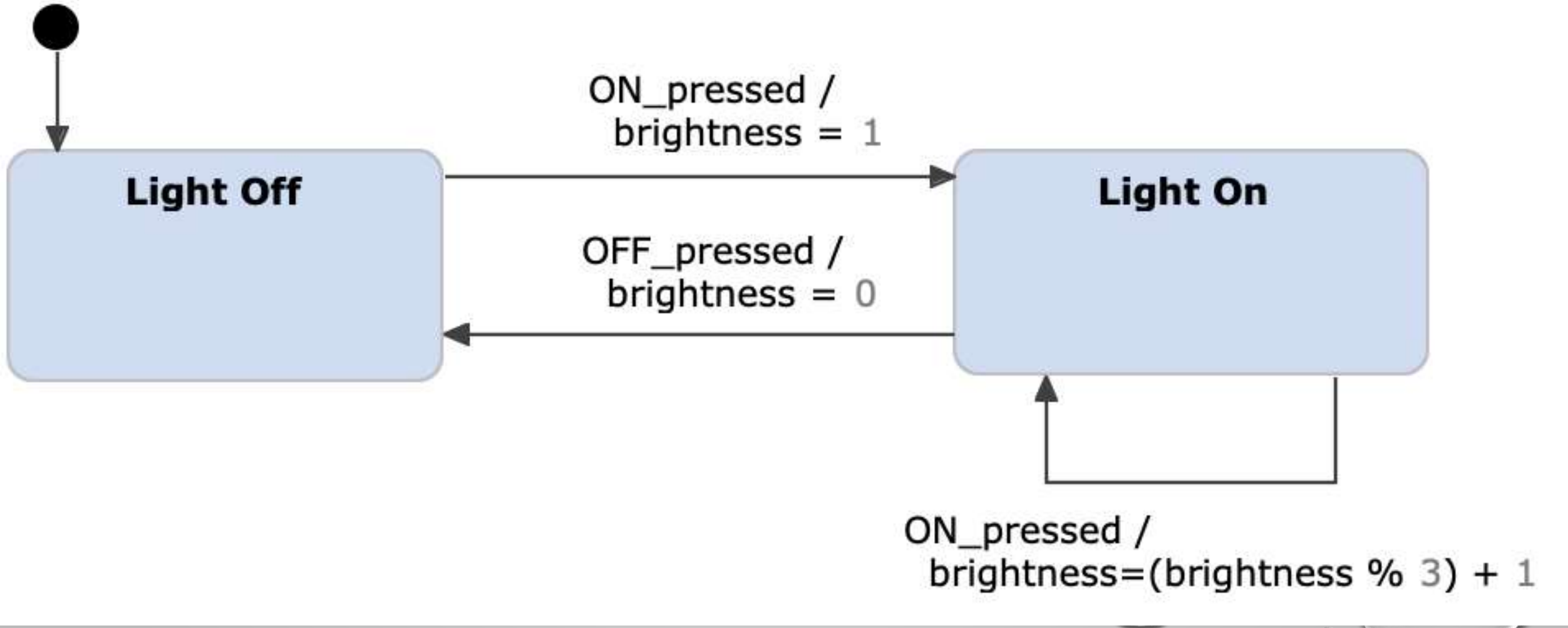
He coined the term “statechart”

The statecharts proposed by him consist of features from both the Mealy machine, Moore machine, and on top of that he adds a lot of other features.

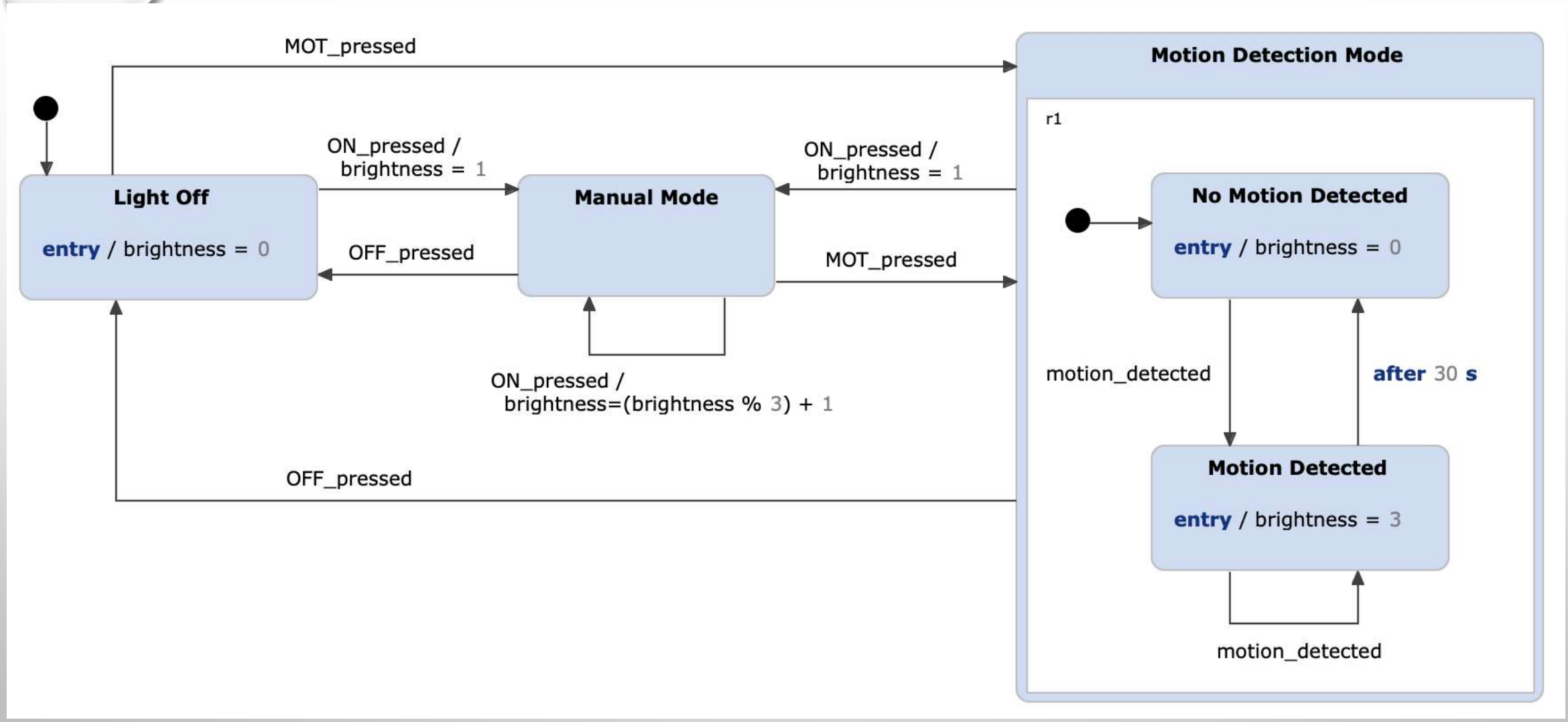
Harel statecharts feature:

- It introduces the concept of substates and superstates
- Composite states
- History states(shallow and deep)
- Orthogonality
- Communication between state machines
- Conditional transitions(guards)
- Entry and exit actions for a state or for a composite state
- Activities inside a state
- Parameterized states
- Overlapping states
- Recursive statecharts

Harel Statechart Example

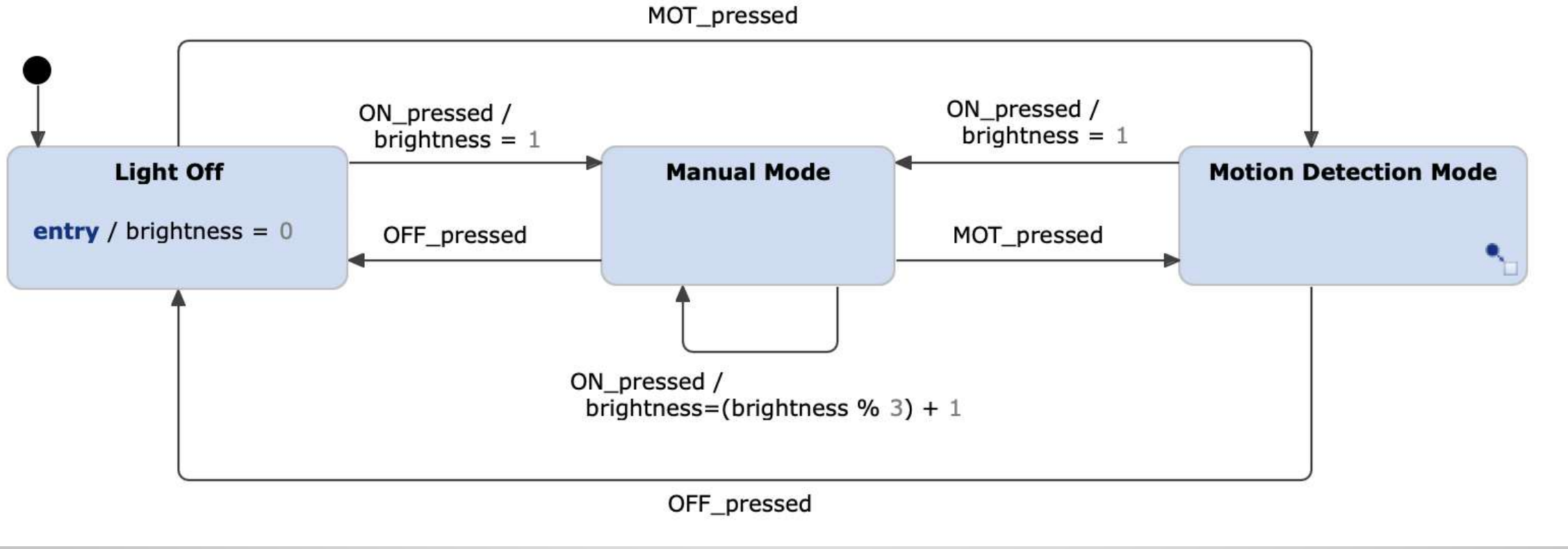


Harel Statechart Example Contd



Harel statechart with composite states

Harel Statechart Example Contd



Harel statechart with sub diagrams

UML State Machine

UML state machine is the present age machines

- UML state machines are based on the statechart notation introduced by David Harel.
- Furthermore, the UML extends the notation of Harel statecharts by object-oriented principles.
- Mapping this to our light switch example, in UML the possible actions of the light switch are model as a type with operations turnOn(), turnOff(), setBrightness(value) and so on.

UML State Machine

A

B

C